

Topic 3: Singly Linked List

3.1 What is a Linked List?

- A sequence of nodes where each node stores data and a pointer to the next node
- Nodes are NOT in contiguous memory -- scattered on the heap, linked by pointers
- Advantages over arrays: $O(1)$ insert/delete at known position, no pre-declared size
- Disadvantages: $O(n)$ random access (must traverse from head), extra memory for pointers

3.2 Node Structure

- Each node has two fields: data (payload) and next (pointer to the next node)
- The last node's next is nullptr -- marks the end of the list
- head pointer: points to the first node; if head == nullptr the list is empty

```
struct Node {  
    int    data;  
    Node* next;  
    Node(int d) : data(d), next(nullptr) {}  
};
```

3.3 Core Operations

- insertFront: create node, set node->next = head, set head = node -- $O(1)$
- insertEnd: traverse to last node, set last->next = newNode -- $O(n)$
- deleteNode(val): find node with val, update previous node's next to skip it, delete node -- $O(n)$
- search(val): traverse from head until data == val or nullptr -- $O(n)$
- printList: traverse from head, print each node's data -- $O(n)$

```

class LinkedList {
    Node* head = nullptr;
public:
    void insertFront(int d) {
        Node* n = new Node(d);
        n->next = head;
        head = n;
    }
    void insertEnd(int d) {
        Node* n = new Node(d);
        if (!head) { head = n; return; }
        Node* cur = head;
        while (cur->next) cur = cur->next;
        cur->next = n;
    }
    void print() {
        for (Node* cur = head; cur; cur = cur->next)
            cout << cur->data << " -> ";
        cout << "NULL" << endl;
    }
};

```

3.4 Reversing a Linked List

- Three pointers: prev (nullptr), curr (head), next
- Iteration: save next = curr->next, reverse link curr->next = prev, advance prev = curr, curr = next
- After loop, head = prev
- Time: $O(n)$, Space: $O(1)$

```

void reverse() {
    Node *prev = nullptr, *curr = head, *next = nullptr;
    while (curr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    head = prev;
}

```